Safe hand-off control of a quadplane VTOL across an uncertain plant family with a history-conditioned transformer

Md Nur-A-Adam Dony[a,*], Md Azizul Islam[b]

[a]*Department of Electrical and Computer Engineering, Tennessee Technological University, Cookeville, TN, 38505, USA*
[b]*KaiLab LLC, Sugar Land, TX, 77479, USA*

Abstract

The hand-off of a quadplane between its lift rotors and its wing decides whether the aircraft flies or falls, and open-source autopilots decide it by an airspeed threshold. This article introduces HOT, a history-conditioned transformer that reads the last L autopilot samples, infers the aircraft it is flying through a context token, and commands the pitch setpoint, the pusher and the rotors through a bounded head. The policy is trained by backpropagation through a differentiable model identified from the flight log of an 8 kg quadplane, under domain randomisation over ten plant parameters, gusts and noise. A lift-safety module rolls a backup manoeuvre out over the vertices of the parameter box and the plants consistent with the observed transitions, and accepts the network command only if every rollout keeps the lift margin, the angle of attack, the flight path and the sink rate within bounds and settles into level flight. Recursive feasibility and constraint satisfaction over the verified plant set are proved, and the context token is shown to resolve the plant parameters while the responses that reveal them are inside the window. On 100 unseen plants, outside the box, under gusts and noise, from states recorded in flight and with the PX4 firmware in the loop, HOT is compared with nine controllers, including model predictive control and reinforcement learning. It completes the front transition in 4.5 s with the lift fraction never below 0.94, keeps every constraint on every unseen plant, and evaluates in 0.6 ms on one core.

Keywords: Neural control, Transformer, Safety filter, Domain randomisation, VTOL aircraft, Physics-informed learning

1. Introduction

A quadplane carries a fixed wing with a pusher propeller and four lift rotors on booms. Between hover and cruise it must accelerate through the airspeed range in which the wing produces little lift, hand the weight of the aircraft from the rotors to the wing and shut the rotors down, and reverse the sequence to land. This hand-off is the phase in which small vertical take-off and landing (VTOL) aircraft are most often lost, and the phase that open-source autopilots handle with the least sophistication: the PX4 stack, which flew the aircraft studied here, ramps the pusher and cuts the rotors when a one-hertz airspeed sample crosses a threshold [1, 2]. The threshold says nothing about the lift actually carried by wing and rotors together, which is the quantity that decides whether the aircraft flies or falls, and it is tuned by hand for one aircraft in one condition.

Learning-based controllers have reached the flight of quadrotors and fixed-wing aircraft [19, 20], and neural networks that imitate or replace model predictive controllers are established [21, 22, 23]. Two obstacles keep them away from the hand-off. First, the plant is uncertain: the polar of a foam wing, the gain and lag of the attitude loop and the maps and lags of two thrust actuators differ from one airframe to the next and from one flight to the next, so a policy trained on one model has no reason to work on another. Second, the hand-off is safety-critical, and a network offers no guarantee on the lift margin, the angle of attack or the sink rate. The safe-learning literature answers the second obstacle with safety filters built on control barrier functions [24], predictive safety filters [25] and backup controllers [26], and the first with domain randomisation [30, 31]; what is missing is a controller that combines both for a plant family with lagged actuators and an attitude loop inside the plant, and that is validated on a flight-identified aircraft against the full range of alternatives.

Novelty. This article treats the hand-off as a learning problem across a family of plants: a single policy must fly every aircraft of an identified parameter box without being told which one it is flying. What is new is (a) a history-conditioned transformer, HOT, whose context token is trained to carry the plant it infers from the last L autopilot samples, so that the policy adapts within a fraction of a second without online identification; (b) a lift-safety module with a proof of recursive feasibility: a backup manoeuvre built from measured quantities only is rolled out from the predicted next state over the vertices of the box and the plants consistent with the observed transitions, the network command being accepted only if every rollout keeps the lift margin, the angle of attack, the flight path and the sink rate within bounds and settles into level flight; and (c) physics-informed training through the differentiable identified model under domain randomisation over the box, gusts and sensor noise, with no expert controller, no reward engineering and no simulator beyond the identified equations. The contributions are: (i) the HOT architecture with its four modules and a bounded head that enforces the input

*Corresponding author.
Email address: mdony42@tntech.edu (Md Nur-A-Adam Dony)

box and the actuator rates by construction (Section 5); (ii) the lift-safety module and its analysis, Lemmas 1–4 and Theorem 1, with a verifiable convergence condition and an identifiability result for the context token (Section 6); (iii) a flight-identified platform, an 8 kg quadplane whose ten-parameter box comes from its PX4 log (Section 7); and (iv) an evaluation against nine controllers, from the autopilot rule to online model predictive control, feed-forward and recurrent policies and two reinforcement-learning agents, on the nominal plant, on 100 unseen plants of the box, outside the box, under gusts and noise, from the states recorded in flight and with the PX4 firmware in the loop, with Wilcoxon and Friedman statistics, ablations, parameter sweeps and attention analyses (Section 8). The flight data are released as a dataset [16]. The code, the trained networks, the results of every experiment and the article sources are available at `https://github.com/AdamDony/quadplane-hot`.

2. Related work

Transition control of hybrid VTOL aircraft. Hierarchical and gain-scheduled controllers [3, 4] blend a multicopter and a fixed-wing loop over the airspeed, optimised manoeuvres [5] plan the transition offline, and model predictive control has flown tail-sitter and tilt-rotor transitions along prescribed trajectories with nominal models [6, 7, 8]. The parametric uncertainty of a VTOL vehicle has been handled by adaptive feedback [9]. None of these controllers learns across a family of airframes or guarantees the lift carried by wing and rotors together.

Learned controllers with safety. Approximate model predictive control trains a network on the solutions of an optimiser and certifies it by statistical validation [21] or by a projection onto the constraints [23, 22]; learning-based model predictive control is reviewed in [28] and safe learning in robotics in [29]. Safety filters minimally modify a learned command through control barrier functions [24], predictive filters with a terminal set [25], backup controllers whose forward flow is checked online [26], or shields synthesised from a model [27]. Robust tracking under parametric uncertainty without learning has been obtained by command-filtered feedback for vessels [10] and by robust formation control of spacecraft [11] and of unmanned aircraft [12], and constraint enforcement by control barrier functions has been combined with learning for cooperative multi-input systems [13], discrete-event systems [14] and saturated linear systems with model-free Q-learning [15]; these works filter or adapt a nominal command rather than learn a policy over a plant family, and none of them treats actuator lags and an inner attitude loop as part of the plant. The module of this article is a backup filter in the sense of [26, 25] extended to a finite plant set contracted online by set-membership, with a settled neighbourhood in place of a terminal invariant set.

Transformers for control and identification. Attention over a window of past observations lets a policy infer the system it is acting on, an idea exploited as in-context system identification and meta-learning [33, 35], and transformers have been trained as controllers by imitation and by reinforcement learning [33, 34]. Domain randomisation [30] and dynamics randomisation [31] train a single policy over a distribution of simulators so that it transfers to the real system; rapid motor adaptation [32] learns an explicit extrinsics encoder from history. HOT follows the same principle with a physics prior: the window is embedded with the dynamic pressure and the lift fraction, the context token is supervised to the plant parameters during training, and the whole is trained through the differentiable model rather than by reward.

Table 1: Notation.

$x = (V, \gamma, \theta, T_c, T_r)$	state	$\vartheta \in \Theta$	plant parameters, box
$u = (\theta_{sp}, T_c^c, T_r^c)$	input	Θ_N, Θ'_k	plant set, consistent subset
$\alpha = \theta - \gamma$	angle of attack	$F(x, u; \vartheta)$	one-step map
$\bar{q},\ L,\ D$	dynamic pressure, lift, drag	$c(x; \vartheta) \in \mathbb{R}^8$	constraint margins
Λ	lift fraction	m	verification margins
$\rho_t,\ x^t(\rho)$	target airspeed, trim	$\kappa_b,\ H,\ C$	backup, horizon, settled set
$\mathcal{X},\ \mathcal{U},\ \mathcal{U}_k$	state, input, admissible sets	$\mathcal{S}(\cdot)$	safe set
$\mathsf{w}_k,\ L$	window, its length	$\pi_\phi,\ \ell_\pi$	policy, Lipschitz constant
$z_{ctx},\ \hat{\vartheta}$	context token, plant estimate	$W(x)$	Lyapunov-like function
$\bar{\delta}_i$	command step bounds	$\dot{\gamma}_{lim},\ K_L$	sink-rate limit, lift deficit

3. Preliminaries

3.1. Notation

Table 1 lists the symbols used throughout. For a symmetric $M \succ 0$, $\|y\|_M = \sqrt{y^\top M y}$; $\mathrm{sat}_{[a,b]}(\cdot)$ is the saturation to $[a, b]$; co is the convex hull; $\mathbf{1}$ the vector of ones. A hat marks an estimate, a bar a bound, the superscript t a trim value and the subscript k the sampling index at period Δ. Angles are in radians in the equations and in degrees in the text.

3.2. Longitudinal flight with the attitude loop inside the plant

Fig. 1 shows the longitudinal motion of a quadplane as a point mass in the vertical plane, described in wind axes [17]: the air-relative speed V, the flight-path angle γ, the pitch θ and the angle of attack $\alpha = \theta - \gamma$; the pusher thrust T_c along the body axis, the rotor thrust T_r normal to it, the lift L and drag D of the wing. Below stall the wing follows the linear lift curve and the parabolic drag polar, $C_L = C_{L0} + C_{L\alpha}\alpha$ and $C_D = C_{D0} + kC_L^2$ with $k = 1/(\pi e\,\mathrm{AR})$ [17, 18]. Small VTOL autopilots keep their attitude loop active in every flight mode, so the plant seen by an outer controller contains that loop; it is represented by a first-order response $\dot{\theta} = \kappa(g_\theta \theta_{sp} - \theta)$ to the pitch setpoint, and the two thrust actuators by first-order lags with a gain error each.

3.3. Backup safety filters

A safety filter takes a candidate command from any controller and returns a command that keeps the state in a safe set. A backup filter [26, 25] defines the safe set through a fixed backup law: a state is safe if the backup trajectory from it satisfies the constraints over a horizon and ends in a set that the backup keeps invariant. The following fact is the shift argument behind such filters.

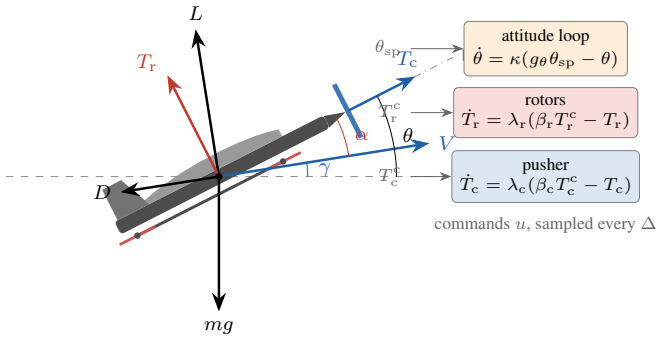


Figure 1: Longitudinal point mass in the wind axes with the wing forces L, D, the pusher thrust T_c along the body axis, the rotor thrust T_r normal to it and the angles γ, θ, α; the attitude loop and the actuators are first-order lags inside the plant.

Fact 1 (Shift argument [25])**.** *Let a trajectory $x^{(0)}, \dots, x^{(H)}$ generated by a backup law satisfy the constraints and end in a set that is forward invariant under the backup law. Then the trajectory generated by the backup law from $x^{(1)}$ satisfies the constraints over H steps and ends in the same set.*

3.4. Self-attention with a readout token

A transformer block maps a sequence of tokens $e_1, \dots, e_n \in \mathbb{R}^d$ to itself through multi-head self-attention and a feed-forward network with residual connections and layer normalisation [36]. A readout token prepended to the sequence attends to every input token and collects a summary of the sequence; its output is used for classification in vision and language models and here for the plant estimate.

Fact 2 (Universal approximation of the readout [37])**.** *Transformer blocks with a readout token can approximate any continuous map from a compact set of token sequences to $\mathbb{R}^p$ uniformly to any accuracy.*

4. Problem formulation

4.1. Plant family

With $\bar q = \frac{1}{2}\rho V^2$, $C_L = C_{L0} + C_{L\alpha}\alpha$, $L = \bar q S C_L$ and $D = \bar q S (C_{D0} + k C_L^2)$, the state $x = (V, \gamma, \theta, T_c, T_r)$ and the input $u = (\theta_{sp}, T_c^c, T_r^c)$ obey

$$m\dot V = T_c\cos\alpha - D - T_r\sin\alpha - mg\sin\gamma, \tag{1a}$$
$$mV\dot\gamma = L + T_c\sin\alpha + T_r\cos\alpha - mg\cos\gamma, \tag{1b}$$
$$\dot\theta = \kappa(g_\theta\theta_{sp} - \theta), \qquad \dot T_i = \lambda_i(\beta_i T_i^c - T_i),\ i \in \{c, r\}. \tag{1c}$$

The parameter vector $\vartheta = (C_{L0}, C_{L\alpha}, C_{D0}, k, \kappa, g_\theta, \lambda_c, \lambda_r, \beta_c, \beta_r)$ is unknown to the controller and lies in the box $\Theta = \prod_i [\underline{\vartheta}_i, \overline{\vartheta}_i]$ identified from the flight log (Section 7, Table 2); it may differ from one airframe or one flight to the next but is constant during a hand-off. The commands are sampled every $\Delta = 50$ ms and held; $F(x, u; \vartheta)$ denotes the exact one-step map of (1). A vertical gust w enters as an angle-of-attack perturbation $\arctan(w/V)$ and the autopilot delivers the state with bounded error $|e_k| \le \varepsilon$.

4.2. Constraints and the lift fraction

The operating region and the input set are

$$\mathcal{X} = \{x :\ V \ge V_{min},\ \gamma \in [\underline\gamma, \overline\gamma],\ \theta \in [\underline\theta, \overline\theta],\ \alpha \le \bar\alpha\}, \tag{2}$$
$$\mathcal{U} = [-\bar\theta_{sp}, \bar\theta_{sp}] \times [0, \overline{T}_c] \times [0, \overline{T}_r],$$
$$\mathcal{U}_k = \{u \in \mathcal{U} :\ |u_i - u_{i,k-1}| \le \bar\delta_i\}, \tag{3}$$

the step bounds $\bar\delta$ expressing the rate limits of the attitude setpoint and of the two thrust commands. The lift fraction

$$\Lambda(x; \vartheta) = \frac{L + T_c\sin\alpha + T_r\cos\alpha}{mg\cos\gamma} \tag{4}$$

is the share of the weight carried by wing and rotors together; the hand-off is safe when $\Lambda \ge 1 - K_L$ at every sampling instant, with K_L the largest transient deficit tolerated, and the sink rate $\dot\gamma$ from (1b) stays above $-\dot\gamma_{lim}$. The eight margins

$$c(x; \vartheta) = (\Lambda - (1 - K_L),\ \bar\alpha - \alpha,\ \overline\theta - \theta,\ \theta - \underline\theta,\ \overline\gamma - \gamma,\ \gamma - \underline\gamma,\ \dot\gamma + \dot\gamma_{lim},\ V - V_{min}) \tag{5}$$

collect the constraints; $c \ge 0$ means all of them hold.

4.3. Objectives

A hand-off is a change of the target airspeed ρ_t from the hover-exit value ρ_0 to the cruise value ρ_c (front transition) or back, with the trim $x^t(\rho_t)$ of the nominal plant as the destination. The controller reads the window $w_k = (x_{k-L+1}, u_{k-L}, \dots, x_k, u_{k-1})$ and the target, and must, for every $\vartheta \in \Theta$:

- P1 keep $c(x_k; \vartheta) \ge 0$ and $u_k \in \mathcal{U}_k$ at every sampling instant (safety for the whole family);
- P2 drive V_k to within ± 0.3 m/s of ρ_t and x_k to a neighbourhood of $x^t(\rho_t)$ (convergence);
- P3 do so with a single set of weights, without being told ϑ and without online identification (generalisation across the family), in a time and with an energy competitive with controllers that know the plant.

5. HOT: architecture, training and the lift-safety module

Fig. 2 shows the controller. Four modules process the window: the sliding-window encoder (SWE) turns the last L samples into tokens with physics features, the plant-inference attention (PIA) mixes them with a context token that carries the plant estimate, the bounded policy head (BPH) turns the context and the current-state token into a command inside the input box and the step bounds, and the lift-safety module (LSM) certifies the command against a backup manoeuvre over the plant family.

5.1. Sliding-window encoder

Each sample (x_j, u_{j-1}) of the window is scaled and augmented with four physics features, the dynamic pressure $\bar q_j$, the wing lift fraction of the nominal polar $L(V_j, \alpha_j; \hat\vartheta)/(mg)$, the lift fraction (4) evaluated with the nominal polar, and α_j; a linear layer maps the 12 numbers to a token $e_j \in \mathbb{R}^d$ and a learned position code is added. A learned context token e_{ctx} is prepended, giving the sequence $(e_{ctx}, e_{k-L+1}, \dots, e_k)$.

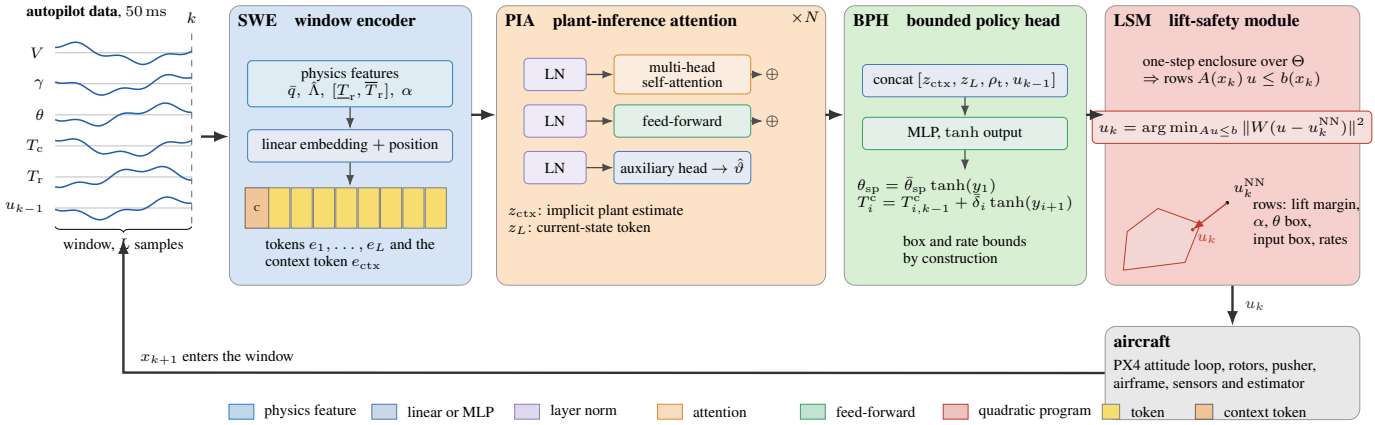


Figure 2: HOT. The window of the last L autopilot samples is embedded with physics features (SWE); a transformer with a context token infers the plant (PIA) and provides the context and the current-state token to the bounded head (BPH), whose outputs respect the input box and the rate bounds by construction; the lift-safety module (LSM) rolls a backup manoeuvre out from the predicted next state over the plant set and returns the network command when every rollout is safe, otherwise the closest certified command towards the backup. The command goes to the attitude loop, the rotors and the pusher of the autopilot, and the next state enters the window.

5.2. *Plant-inference attention*

N transformer blocks, each a multi-head self-attention layer and a feed-forward layer with residual connections and pre-layer normalisation [36], act on the sequence. The output at the context position, z_{ctx}, and at the last position, z_L, are read out. An auxiliary head maps z_{ctx} to a plant estimate inside the box,

$$\hat{\vartheta} = \underline{\vartheta} + (\overline{\vartheta} - \underline{\vartheta}) \odot \sigma(\mathrm{MLP}_{aux}(z_{ctx})), \tag{6}$$

supervised during training to the sampled parameters; it is not used by the command at run time, but it makes the context token an interpretable, testable estimate (Proposition 1 and Section 8.9).

5.3. *Bounded policy head*

The head receives z_{ctx}, z_L, the target ρ_t and the previous command u_{k-1} and returns

$$u_k^{NN} = \mathrm{sat}_{\mathcal{U}}(u_{k-1} + \bar{\delta} \odot \tanh(\mathrm{MLP}(z_{ctx}, z_L, \rho_t, u_{k-1}))), \tag{7}$$

so that $u_k^{NN} \in \mathcal{U}_k$ for every value of the weights: the box and the step bounds of all three commands, the pitch setpoint included, hold by construction. The last layer is initialised at zero, so training starts from the hold policy $u_k = u_{k-1}$.

5.4. *Lift-safety module*

The module uses a backup law κ_b built from measured quantities only: the flight-path angle, the measured flight-path rate $\dot{\gamma}_m$ and acceleration $\dot{V}_m$, the airspeed and the previous command,

$$\theta_{sp} = \gamma + \tau_b \min\{r(\gamma), 0\} + \alpha_b(V), \qquad r(\gamma) = \mathrm{sat}_{[\underline{r},\overline{r}]}(-k_\gamma \gamma), \tag{8a}$$

$$T_r^c = T_{r,k-1}^c + \mathrm{sat}_{[-\bar{\delta}_r,\bar{\delta}_r]}(k_r(r(\gamma) - \dot{\gamma}_m)), \tag{8b}$$

$$T_c^c = T_{c,k-1}^c + \mathrm{sat}_{[-\bar{\delta}_c,\bar{\delta}_c]}(k_c(a(V) - \dot{V}_m)), \tag{8c}$$

$$a(V) = \mathrm{sat}_{[-\bar{a},\bar{a}]}(k_V(\mathrm{proj}_{[\underline{V}_b,\overline{V}_b]}(V) - V)),$$

each saturated to the input box: the pitch setpoint places the wing at the nominal-trim angle of attack of the current airspeed $\alpha_b(V)$ and, when levelling off from a climb, leads the flight path by τ_b seconds of the reference rate $r(\gamma)$, so that a slow attitude loop does not overshoot the angle of attack while the rotors flatten the flight path; the rotors act as a rate-limited integral controller that drives the flight-path rate to the level-off reference; the pusher is the same kind of controller on the measured acceleration, with a reference that is zero inside the airspeed corridor $[\underline{V}_b, \overline{V}_b]$ and pulls the airspeed back from outside, so the backup holds the current airspeed rather than accelerating through the corridor. Nothing in (8) depends on ϑ. Let $\Theta_N \subset \Theta$ be a finite plant set containing the vertices of the box and a sample of its interior, $m \in \mathbb{R}^8_{>0}$ verification margins and C the settled neighbourhood of level flight, a band of lift fraction, flight-path angle and rate, angle of attack, airspeed and acceleration.

Definition 1 (Certified command, safe set)**.** *For x, the previous command u^-, a candidate u and a plant ϑ, the backup rollout is $x^{(0)} = F(x, u; \vartheta)$, $x^{(j+1)} = F(x^{(j)}, \kappa_b(x^{(j)}, u^{(j)}); \vartheta)$ with $u^{(j)}$ the backup commands. The candidate is certified on $\Theta' \subseteq \Theta_N$ if*

$$c(x^{(j)}; \vartheta) \ge m \;\; (j = 0, \ldots, H_j), \qquad x^{(H_j)} \in C \qquad \forall \vartheta \in \Theta', \tag{9}$$

where $H_j \le H$ is the first step at which every plant is settled. The safe set is $\mathcal{S}(\Theta') = \{(x, u^-) :\ \kappa_b(x, u^-)$ is certified on $\Theta'\}$.

The module returns the network command when it is certified on the current plant set Θ'_k, and otherwise the certified point closest to it on the segment towards an anchor u_a, the hold command u_{k-1} when that is certified (the mildest intervention) and the backup command otherwise,

$$u_k = u_a + \lambda^\star(u_k^{NN} - u_a), \qquad \lambda^\star = \max\{\lambda \in [0, 1] :\ u_a + \lambda(u_k^{NN} - u_a) \text{ certified on } \Theta'_k\}, \tag{10}$$

found by bisection, with $u_b = \kappa_b(x_k, u_{k-1})$ the backup command; when not even u_b is certified on Θ'_k, the backup command is

Algorithm 1 Physics-informed training of HOT

build the differentiable model F of (1) and the box Θ; initialise ϕ with the hold policy (7)
for iteration = $1, \dots, I$ **do**
  $N \leftarrow$ curriculum length; sample plants $\vartheta \sim \mathcal{U}(\Theta)$, trims ρ_0, targets ρ_t, gusts, noise
  roll out $x_{j+1} = F(x_j, \pi_\phi(\mathsf{w}_j + \text{noise}); \vartheta) + w_j$, $j < N$, freezing diverged episodes
  $\phi \leftarrow \phi - \eta \nabla_\phi \mathcal{J}(\phi)$ by truncated backpropagation through time, gradient norm clipped
end for
verify the decrease condition of Definition 2 on the grid; compute the Lipschitz bound (16)

applied: it remains certified on the true plant, whose rollout is the tail of the previous certificate (Lemma 3), even when other retained plants fail from the current state; these applications are counted in Section 8. The plant set is contracted online by set-membership with the measurement bound ε,

$$\Theta'_{k+1} = \{\vartheta \in \Theta'_k : \ |F(x_k, u_k; \vartheta) - x_{k+1}| \le \varepsilon\}, \qquad \Theta'_0 = \Theta_N, \tag{11}$$

so that plants inconsistent with the observed motion stop constraining the command. Algorithm 2 lists one sampling period.

5.5. Physics-informed training

The policy is trained by backpropagation through rollouts of the differentiable model (1), Fig. 3 and Algorithm 1. Each episode draws a plant $\vartheta \sim \mathcal{U}(\Theta)$, an initial trim ρ_0, a target ρ_t (front, back or an arbitrary pair), a gust profile (first-order coloured noise plus an offset) and sensor noise on the window; the rollout $x_{j+1} = F(x_j, \pi_\phi(\mathsf{w}_j); \vartheta) + w_j$ runs for N steps, N growing from 3 s to 15 s over the training (curriculum), and the loss is

$$\begin{aligned}\mathcal{J}(\phi) = \sum_j \Big[& \sigma\Big(\tfrac{|V_j - \rho_t| - 0.3}{0.2}\Big) + |V_j - \rho_t| + w_E \tfrac{P(T_{c,j}, T_{r,j})}{P_0} \\ & + w_B \sum_i \operatorname{softplus}\Big(-\tfrac{c_i(x_j;\vartheta)}{\epsilon_i}\Big) + w_A \|\hat{\vartheta}_j - \vartheta\|^2_{\Sigma^{-1}} \\ & + w_S \big(\|u_j - u_{j-1}\|^2_R + \operatorname{softplus}\big(\tfrac{|\gamma_j| - \gamma_0}{\gamma_1}\big)\big)\Big]\Delta,\end{aligned} \tag{12}$$

a time term (the soft indicator of not yet being at the target plus the distance to it, which carries a gradient far from the target), the energy of the identified power model, a soft barrier on the eight margins, the auxiliary plant loss, and a smoothness term with a preference for level flight ($\gamma_0 = 4°$, $\gamma_1 = 2°$) that keeps the learned hand-off from trading height for speed. Gradients flow through the model and the policy; backpropagation is truncated every 100 steps and the window inputs carry the noise of the estimator. The safety module is not in the training loop, so the network learns to be safe by itself and the module intervenes at run time only where the learned policy would not be; the intervention rate is reported.

Algorithm 2 One sampling period of HOT with the lift-safety module

receive x_k, $\dot{\gamma}_{m,k}$; append (x_k, u_{k-1}) to the window; contract Θ'_k by (11)
$u_k^{NN} \leftarrow \pi_\phi(\mathsf{w}_k)$ by (7)
if u_k^{NN} is certified on Θ'_k by (9) **then** $u_k \leftarrow u_k^{NN}$
else $u_b \leftarrow \kappa_b(x_k, u_{k-1})$ by (8); $u_k \leftarrow$ (10) by bisection (u_b if none certified, Lemma 3)
send θ_{sp} and the rotor collective as an attitude target and the pusher command as an actuator command

6. Analysis

Let Θ_N, m, C, H and ε be as in Section 5.4, κ_b the backup law (8) and F the one-step map. All proofs are in the main text.

6.1. The backup law

Lemma 1 (Rotor loop of the backup)**.** *For a frozen wing state (V, α) with $V \ge V_{min}$, the loop formed by the rotor lag* (1c)*, the flight-path rate* (1b) *and the integral law* (8b) *is exponentially stable for every (λ_r, β_r) of the box if*

$$g_{max} = k_r \frac{\overline{\beta}_r (1 - e^{-\overline{\lambda}_r \Delta})}{m V_{min}} < 1, \tag{13}$$

and, when the lag gap vanishes and the rate limit $\bar{\delta}_r$ is inactive, the flight-path rate converges monotonically to the reference $r = \operatorname{sat}_{[\underline{r}, \bar{r}]}(-k_\gamma \gamma)$.

Proof. With the wing frozen, $\dot{\gamma}_k = q\, T_{r,k} + \ell$ with $q = \cos\alpha/(mV)$ and a constant ℓ. Let $e_k = r - \dot{\gamma}_k$, $a = e^{-\lambda_r \Delta}$ and $s_k = \beta_r T^c_{r,k-1} - T_{r,k}$ the lag gap. The exact step of the lag and the law (8b), $T^c_{r,k} = T^c_{r,k-1} + k_r e_k$, give

$$\begin{aligned} T_{r,k+1} &= a T_{r,k} + (1-a)\beta_r T^c_{r,k} \\ &= T_{r,k} + (1-a)(s_k + \beta_r k_r e_k), \qquad (14) \\ e_{k+1} &= e_k - q\,(T_{r,k+1} - T_{r,k}) = (1-g)\, e_k - q(1-a)\, s_k, \\ s_{k+1} &= \beta_r T^c_{r,k} - T_{r,k+1} = a\, s_k + a\beta_r k_r\, e_k, \qquad (15) \end{aligned}$$

with $g = q(1-a)\beta_r k_r$, a linear recursion in (e_k, s_k) with matrix $M = \left[\begin{smallmatrix} 1-g & -q(1-a) \\ a\beta_r k_r & a \end{smallmatrix}\right]$, $\operatorname{tr} M = 1 - g + a$ and $\det M = a(1-g) + ag = a$. The Jury conditions $|\det M| < 1$, $1 - \operatorname{tr} M + \det M = g > 0$ and $1 + \operatorname{tr} M + \det M = 2 - g + 2a > 0$ hold for $0 < g < 1$ and $0 < a < 1$, so both eigenvalues lie inside the unit disc. Since $q \le 1/(mV_{min})$ and $(1-a)\beta_r \le (1 - e^{-\overline{\lambda}_r \Delta})\overline{\beta}_r$, $g \le g_{max}$, and (13) gives $0 < g < 1$ for every plant of the box. With $s_k = 0$ the recursion reduces to $e_{k+1} = (1-g) e_k$, monotone convergence. □

With the values of Section 7, $g_{max} = 0.17$. The pusher law (8c) has the same structure with $(\lambda_c, \beta_c, k_c)$ and $q = \cos\alpha/m$, so the same recursion and the condition $k_c \overline{\beta}_c (1 - e^{-\overline{\lambda}_c \Delta})/m < 1$ (= 0.17 here) give monotone convergence of the acceleration to its reference. The pitch law (8a) keeps the wing at the trim angle of attack of the current airspeed, so the wing carries what it can and the rotors carry the rest; the pusher law keeps the airspeed in the corridor. Their interaction with the rotor loop is covered by the settling property below.

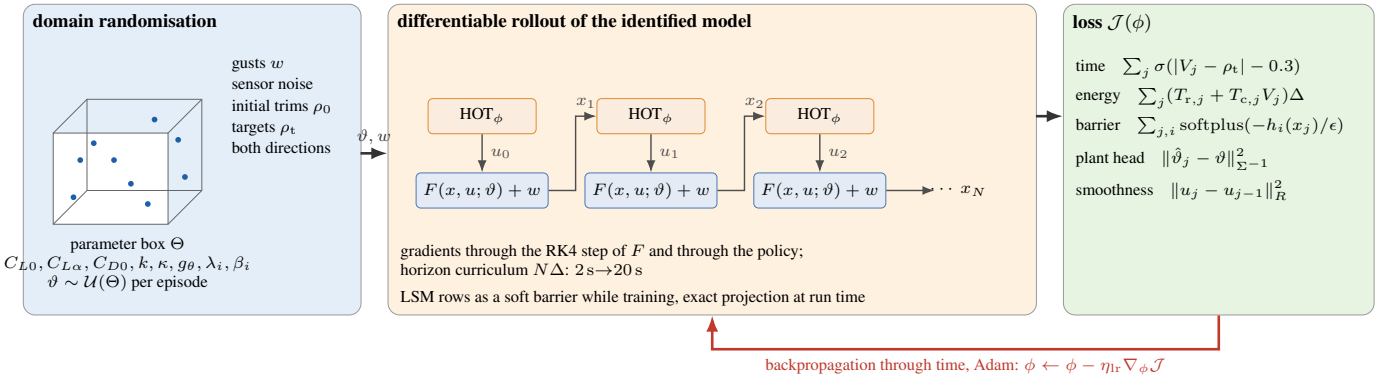


Figure 3: Physics-informed training: domain randomisation over the box, gusts, noise, trims and targets; differentiable rollout of the identified model with the policy in the loop; loss terms; backpropagation through time with a horizon curriculum.

Assumption 1 (Settled neighbourhood)**.** *For every $\vartheta \in \Theta_N$ and every $x \in C$ with $c(x;\vartheta) \geq m$, the backup closed loop from x satisfies $c(x^{(j)};\vartheta) \geq 0$ for all $j \geq 0$.*

Assumption 1 states that the backup keeps level flight once it has reached it. It is verified numerically over $C \times \Theta$ by rollouts of 20 s from 23,914 sampled (state, plant) pairs over the full vertex set and interior samples of the box, all of which keep the constraints with the margin m (0 failures; the smallest margin of the lift constraint, 0.011 in lift-fraction units, is the tightest, Section 7.3); the margin m absorbs the sampling of C through the Lipschitz constant of c along the backup flow [44].

6.2. The lift-safety module

Lemma 2 (The true plant is never discarded)**.** *If $\vartheta \in \Theta_N$ is the true plant and $|e_k| \leq \varepsilon$, then $\vartheta \in \Theta'_k$ for all k.*

Proof. $x_{k+1} = F(x_k, u_k; \vartheta) + e_k$ with $|e_k| \leq \varepsilon$, so the test in (11) holds for ϑ at every step; induction from $\Theta'_0 = \Theta_N$. □

Lemma 3 (Recursive feasibility)**.** *Let the true plant $\vartheta \in \Theta_N$, Assumption 1 hold, $|e_k| \leq \varepsilon$ and $(x_0, u_{-1}) \in \mathcal{S}(\Theta_N)$. Then at every step the command returned by* (10) *is certified on $\{\vartheta\}$ and $(x_{k+1}, u_k) \in \mathcal{S}(\{\vartheta\})$.*

Proof. Induction on k; suppose $(x_k, u_{k-1}) \in \mathcal{S}(\{\vartheta\})$, i.e. the backup rollout $x^{(0)}, \ldots, x^{(H_j)}$ from x_k with u_b satisfies (9) for ϑ. If the module returns a candidate certified on Θ'_k, then, since $\vartheta \in \Theta'_k$ by Lemma 2, (9) holds for ϑ from $x_{k+1} = x^{(0)}$ of that candidate's rollout, which is $(x_{k+1}, u_k) \in \mathcal{S}(\{\vartheta\})$. If no candidate is certified on Θ'_k the module returns u_b, so $x_{k+1} = x^{(0)}$ of the rollout verified at step k; the backup rollout from x_{k+1} is its tail $x^{(1)}, \ldots, x^{(H_j)}$, which satisfies $c \geq m$ and ends in C by (9) (Fact 1), and by Assumption 1 the closed loop continues from $x^{(H_j)} \in C$ with $c \geq 0$; hence u_b is certified on $\{\vartheta\}$ and $(x_{k+1}, u_k) \in \mathcal{S}(\{\vartheta\})$. □

Lemma 4 (Extension to the box)**.** *Let ℓ_ϑ be a Lipschitz constant of $\vartheta \mapsto \min_j c(x^{(j)};\vartheta)$ over the rollouts of Definition 1 and $d_N = \max_{\vartheta \in \Theta} \min_i \|\vartheta - \vartheta_i\|$ the covering radius of Θ_N. A candidate certified on Θ_N with margin $m \geq \ell_\vartheta d_N \mathbf{1}$ is certified with margin 0 on the whole box.*

Proof. For $\vartheta \in \Theta$ take ϑ_i with $\|\vartheta - \vartheta_i\| \leq d_N$; then $\min_j c(x^{(j)};\vartheta) \geq \min_j c(x^{(j)};\vartheta_i) - \ell_\vartheta d_N \geq m - \ell_\vartheta d_N \geq 0$. □

6.3. The policy

Lemma 5 (Lipschitz bound)**.** *Let each attention block have H_h heads with matrices W_Q^h, W_K^h, W_V^h, W_O, token norms bounded by r on the operating windows, and each feed-forward block weights W_1, W_2 with a 1-Lipschitz activation. Then π_ϕ is Lipschitz on the operating windows with*

$$\ell_\pi \leq \|\bar{\delta}\|_\infty \, \|W_{\mathrm{mlp}}\|_{\mathrm{prod}} \, \|W_{\mathrm{emb}}\|_2 (1 + \|W_2\|_2 \|W_1\|_2) \cdot \prod_{n=1}^{N} \Big(1 + \|W_O\|_2 \sum_h \|W_V^h\|_2 \big(1 + \tfrac{2r^2 \|W_Q^h\|_2 \|W_K^h\|_2}{\sqrt{d_h}}\big)\Big), \tag{16}$$

where $\|\cdot\|_{\mathrm{prod}}$ is the product of the spectral norms of the head's layers.

Proof. A residual block with an ℓ-Lipschitz branch is $(1+\ell)$-Lipschitz. The feed-forward branch has $\ell \leq \|W_2\|_2 \|W_1\|_2$. On inputs of norm at most r a self-attention branch is Lipschitz with constant $\|W_O\|_2 \sum_h \|W_V^h\|_2 (1 + 2r^2 \|W_Q^h\|_2 \|W_K^h\|_2 / \sqrt{d_h})$, the second term bounding the sensitivity of the softmax weights, whose Jacobian has spectral norm at most $1/2$, to the scores, which are $2r\|W_Q^h\|_2 \|W_K^h\|_2 r / \sqrt{d_h}$-Lipschitz in the tokens. Layer normalisation is absorbed in r; the embedding is linear; tanh and the saturation are 1-Lipschitz, scaled by the step bounds. Composition multiplies the constants. □

Definition 2 (Verified decrease)**.** *With $W(x) = \|x - x^{\mathrm{t}}(\rho_t)\|_P^2$, $P \succ 0$, and $u(x)$ the filtered network command, the trained closed loop satisfies the decrease condition with constants (c, β, d) if on a grid $\mathcal{G}$ of spacing d covering $\{x \in \mathcal{X} : W(x) \geq \beta\}$ and for every $\vartheta \in \Theta_N$, $W(F(x_i, u(x_i);\vartheta)) - W(x_i) \leq -cW(x_i) - \ell_W d$, with ℓ_W a Lipschitz constant of $x \mapsto W(F(x, u(x);\vartheta)) - W(x)$ obtained from the constants of F, ℓ_π and the filter map.*

6.4. Main result

Theorem 1 (Safe generalised hand-off)**.** *Let the true plant $\vartheta \in \Theta_N$, Assumption 1 hold, $|e_k| \leq \varepsilon$ and $(x_0, u_{-1}) \in \mathcal{S}(\Theta_N)$. Then, for every command sequence of the network,*

(i) (safety) $c(x_k;\vartheta) \geq 0$ *and* $u_k \in \mathcal{U}_k$ *for all* $k \geq 0$*: the lift fraction stays above* $1 - K_L$*, the angle of attack below* $\bar{\alpha}$*, the pitch and the flight path in their boxes and the sink rate above* $-\dot{\gamma}_{\lim}$*; the module always holds a certified command;*

(ii) (practical convergence) if Definition 2 holds with (c, β, d) *and the disturbance is bounded by* $\bar{w}$*, then*

$$W(x_k) \leq \max\{(1-c)^k W(x_0),\ \beta\} \quad (\bar{w} = 0),$$
$$\limsup_k W(x_k) \leq \max\left\{\beta,\ \frac{2\|P\|_2 \bar{w}\, \bar{r} + \|P\|_2 \bar{w}^2}{c}\right\} \quad (\bar{w} > 0), \tag{17}$$

with $\bar{r} = \max_{x \in \mathcal{X}} \|x - x^{\mathrm{t}}\|$*;*

(iii) (intervention) $u_k = u_k^{\mathrm{NN}}$ *whenever the network command is certified on* Θ'_k*;*

(iv) (whole box) if every certificate carries the margin of Lemma 4, (i) holds for every $\vartheta \in \Theta$*.*

Proof. (i) By Lemma 3 the command is certified on $\{\vartheta\}$ at every step, so $c(x_{k+1};\vartheta) = c(x^{(0)};\vartheta) \geq m > 0$ by (9). Both u_k^{NN}, by (7), and the anchor, u_{k-1} trivially and u_{b} by the saturations of (8), lie in the convex set $\mathcal{U}_k$, hence so does the convex combination (10). The module holds u_{b} by Lemma 3. (ii) On the cell of $x_i \in \mathcal{G}$, $W(x_{k+1}) - W(x_k) \leq -cW(x_k) - \ell_W d + \ell_W \|x_k - x_i\| \leq -cW(x_k)$ whenever $W(x_k) \geq \beta$, so $W(x_{k+1}) \leq (1-c)W(x_k)$ until the level β is reached, after which $W(x_{k+1}) \leq (1-c)\beta \leq \beta$; this is the first bound of (17). With disturbances, $W(F + w) - W(F) = 2(F - x^{\mathrm{t}})^{\top} P w + w^{\top} P w \leq 2\|P\|_2 \bar{r}\bar{w} + \|P\|_2 \bar{w}^2 =: \Delta_w$, so $W(x_{k+1}) \leq (1-c)W(x_k) + \Delta_w$, whose iterates satisfy $\limsup_k W(x_k) \leq \Delta_w / c$ when this exceeds β, which is the second bound. (iii) A certified u_k^{NN} gives $\lambda^\star = 1$ in (10). (iv) Lemma 4 applied to every certificate used in Lemma 3. □

Proposition 1 (Identifiability from the window)**.** *On a window of L samples in which* α *and* C_L^2 *are not constant, the aerodynamic parameters* $(C_{L0}, C_{L\alpha}, C_{D0}, k)$ *enter the finite-difference residuals of* (1a)–(1b) *linearly through the regressors* $\bar{q}S\,(1, \alpha)$ *and* $\bar{q}S\,(1, C_L^2)$*; the least-squares estimate from the window is unique, and its error is bounded by* $\sigma\sqrt{L}/\sigma_{\min}(\Phi_L)$ *for measurement noise of standard deviation* σ *and regressor matrix* Φ_L*. The attention blocks with the context token can represent this estimator to any accuracy on compact windows, which is the target of the auxiliary head* (6)*.*

Proof. Rearranging (1b) and (1a), $mV\dot{\gamma} - T_{\mathrm{c}} \sin\alpha - T_{\mathrm{r}} \cos\alpha + mg\cos\gamma = \bar{q}S\,(C_{L0} + C_{L\alpha}\alpha)$ and $-m\dot{V} + T_{\mathrm{c}} \cos\alpha - T_{\mathrm{r}} \sin\alpha - mg\sin\gamma = \bar{q}S\,(C_{D0} + kC_L^2)$, linear in the parameters with the stated regressors; stacking L samples gives $\Phi_L \vartheta_{\mathrm{a}} = y_L + e$, whose least-squares solution is unique iff $\operatorname{rank}\Phi_L = 4$, i.e. α and C_L^2 vary over the window, and $\|\hat{\vartheta}_{\mathrm{a}} - \vartheta_{\mathrm{a}}\| \leq \|\Phi_L^{\dagger}\|\|e\| \leq \sigma\sqrt{L}/\sigma_{\min}(\Phi_L)$. The representability is Fact 2. □

7. Experimental platform

7.1. Aircraft and autopilot

The aircraft (Fig. 4) is an 8 kg twin-boom quadplane with a 2 m foam-core wing of Selig S1223 section [18] (chord 0.292 m, aspect ratio 6.85), four brushless lift motors at the boom corners, an H-tail and a tractor cruise motor with a folding propeller, flown by a Pixhawk-class flight controller running PX4 [1]. The autopilot's estimator provides airspeed, flight-path angle, pitch and their rates at 100 Hz; its multicopter attitude and rate loops stay active through the hand-off and receive the pitch setpoint and the rotor collective as an attitude target, while the pusher receives its command through an actuator-set message.

Table 2: Nominal parameters and the box Θ identified from the flight log.

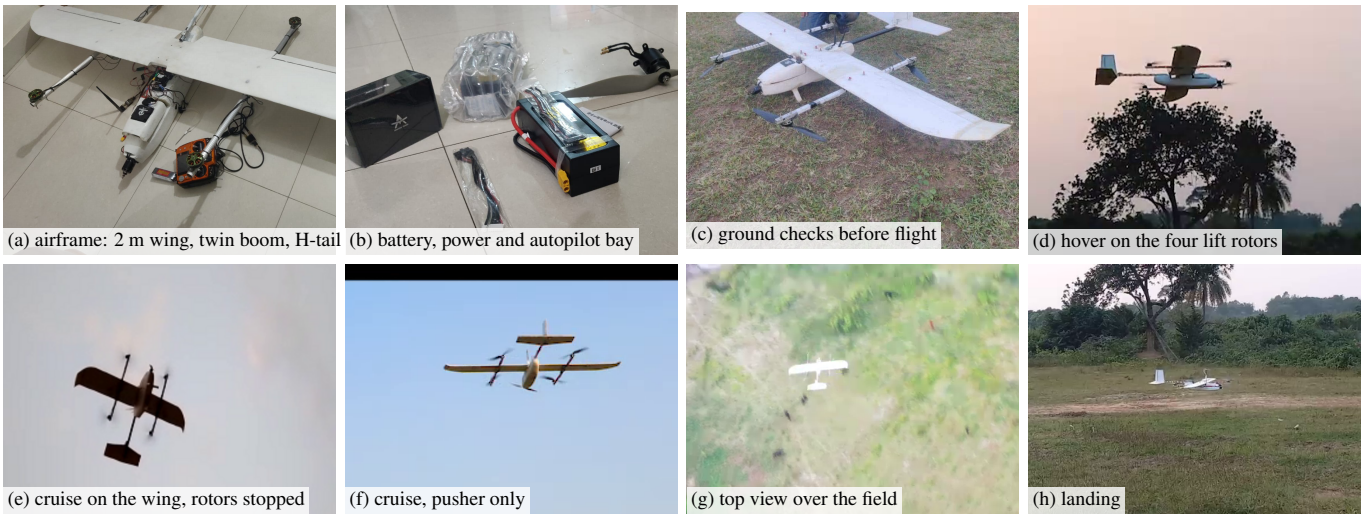
parameter	nominal	interval	parameter	nominal	interval
C_{L0}	1.141	±0.144	κ [1/s]	2.0	[0.67, 5.0]
$C_{L\alpha}$ [1/rad]	3.12	±25%	g_θ	1.0	[0.9, 1.0]
C_{D0}	0.133	±0.044	λ_{c} [1/s]	2.0	[1.33, 4.0]
k	0.060	±25%	λ_{r} [1/s]	6.67	[4.44, 13.3]
$\overline{T}_{\mathrm{c}}$ [N]	30.3	$\beta_{\mathrm{c}} \in [0.78, 1.22]$	$\overline{T}_{\mathrm{r}}$ [N]	118.7	$\beta_{\mathrm{r}} \in [0.85, 1.15]$

7.2. Identified model and parameter box

The parameters of (1) were identified from the PX4 log of a flight with several hand-offs, released as a dataset [16]: the polar $(C_{L0}, C_{L\alpha}, C_{D0}, k)$ from the lift and drag balances of the cruise segments anchored to the computational-fluid-dynamics polar of the wing, the attitude-loop rate and gain (κ, g_θ) from the pitch response to the setpoint, the thrust maps $(k_{\mathrm{c}}, k_{\mathrm{r}})$ from the throttle-to-acceleration balances and the actuator lags from the transitions. Table 2 lists the nominal values and the intervals that define the box Θ: two standard deviations of the identification for the polar, the range of attitude-loop settings of the autopilot for κ and g_θ, and ±50% of the lags and the identified spread of the thrust maps for the actuators. The power of the rotors follows the momentum-theory law $P_{\mathrm{r}} = c_{\mathrm{r}} T_{\mathrm{r}}^{3/2}$ through the measured hover power and the pusher power the linear law $P_{\mathrm{c}} = c_{\mathrm{c}} T_{\mathrm{c}}$ through the measured cruise power. The constraints are $\bar{\alpha} = 5°$ (inside the range verified unstalled in cruise), $\theta \in [-15, 20]°$, $\gamma \in [-15, 15]°$, $V_{\min} = 4$ m/s, $K_L = 0.25$, $\dot{\gamma}_{\lim} = 0.3$ rad/s, the input box $|\theta_{\mathrm{sp}}| \leq 30°$, $T_{\mathrm{c}}^{\mathrm{c}} \in [0, 30.3\,\mathrm{N}]$, $T_{\mathrm{r}}^{\mathrm{c}} \in [0, 118.7\,\mathrm{N}]$ and the step bounds $\bar{\delta} = (4°, 0.75\,\mathrm{N}, 4\,\mathrm{N})$ per 50 ms. The corridor runs from the hover-exit airspeed $\rho_0 = 6$ m/s to the cruise airspeed $\rho_{\mathrm{c}} = 15$ m/s.

7.3. Simulator, training and the safety module

The differentiable model integrates (1) with a Runge–Kutta step of two substeps per sample in PyTorch [45] and the linear polar; the evaluation plant uses five substeps in NumPy and folds the lift curve beyond the stall angles of 10° and −14° with a post-stall slope of −2 rad and a drag rise, so that a policy that leaves the linear range is punished by the plant rather than rewarded by the model. HOT has $L = 32$ tokens of width $d = 48$, $N = 2$ blocks of 4 heads, 79,101 parameters, and is trained for 800 iterations of batch 32 with Adam, learning rate 2×10^{-4} with cosine decay, gradient norm clipped at 1, weights $(w_E, w_B, w_A, w_S) = (0.2, 2, 1, 0.3)$ in (12), gust standard deviation 1 m/s and the noise of Table 3 on the window; the curriculum grows the horizon from 3 s to 15 s over the first 60% of the iterations. The lift-safety module uses the plant set of the 128 vertices of the fast parameters and the



(a) airframe: 2 m wing, twin boom, H-tail (b) battery, power and autopilot bay (c) ground checks before flight (d) hover on the four lift rotors

(e) cruise on the wing, rotors stopped (f) cruise, pusher only (g) top view over the field (h) landing

Figure 4: The quadplane: airframe, power and autopilot bay, ground checks, hover on the lift rotors, cruise on the wing with the rotors stopped, cruise on the pusher, top view over the field, and landing.

Table 3: Sensor-noise standard deviations applied to the window during training and in the noise cell.

	V [m/s]	γ [deg]	θ [deg]	T_c [N]	T_r [N]
level 1	0.3	0.5	0.3	0.6	2.4

lift-curve offset with 32 interior samples online, the backup gains $k_\gamma = 0.4$, $k_r = 10$ N per rad/s per step, the reference band $[-0.15, 0.15]$ rad/s, the lead $\tau_b = 0.75$ s, the pusher loop $k_c = 6$ N per m/s^2 per step, $k_V = 0.5$ /s, $\bar{a} = 1.0$ m/s^2 on the corridor [6, 15.5] m/s, the horizon $H = 200$ samples, the margins $m = (0.02,\ 0°,\ 0°,\ 0°,\ 1°,\ 1°,\ 0.02,\ 0.3)$, the settled neighbourhood $\mathcal{C}$: lift fraction in $[0.93, 1.07]$, $|\gamma| \le 3°$, $|\dot{\gamma}| \le 0.05$ rad/s, $\alpha \in [-9, 0.5]°$, $V \in [5, 16]$ m/s and $|\dot{V}| \le 0.6$ m/s^2, and the measurement bound $\varepsilon = (0.4$ m/s, 1°, 1°, 1.5 N, 4 N). Assumption 1 was verified over the full plant set of 1088 plants; the one-step gain of Lemma 1 is 0.17.

7.4. PX4 firmware in the loop

The autopilot firmware (PX4 v1.17) runs in software-in-the-loop with its simulation-in-hardware plant configured to the identified aircraft: mass, inertia, rotor thrust and lag, the wing with the polar of Table 2, the tail and the pusher lag. The firmware's estimator, attitude and rate loops, control allocation and offboard interface are unchanged; HOT runs as an offboard node that receives the states over MAVLink at 100 Hz, sends the pitch setpoint and the rotor collective as an attitude target and the pusher command as an actuator-set command, and re-plans every 50 ms. The network evaluates in 0.6 ms and the safety check takes 28 ms on average (37 ms at most) in interpreted code on one core, so the loop runs at real time on average and the simulation is slowed to half speed to keep every update inside its period.

8. Experiments

8.1. Set-up, baselines, metrics and statistics

Every controller runs in the same loop: the plant (1) with the true parameters, the state delivered with the noise of Table 3 scaled by the noise level of the cell, the commands sampled every 50 ms and rate-limited, 20 s of simulation. Table 4 lists the nine controllers. The threshold rule (THR) reproduces the autopilot logic; the fixed lift-sharing schedule (SCH) follows the nominal trim of the current airspeed with a pusher loop on the airspeed and a rotor loop on the flight-path rate; the gain-scheduled PID (GS-PID) schedules its gains on the airspeed; the model predictive controller (MPC) solves a nominal-model tracking problem on a 4 s horizon by sequential conic programming; the feed-forward (MLP) and recurrent (LSTM) policies use the bounded head of HOT and are trained by the same physics-informed procedure; HOT−LSM is the transformer without the safety module; PPO [38] and SAC [39] are trained with Stable-Baselines3 [40] on the same plant family with the negative of (12) as the reward. Metrics: the time to the target airspeed (first entry within 0.3 m/s), the energy of the identified power model, the minimum lift fraction, the maximum angle of attack, the extreme flight-path angles and the minimum sink rate, the percentage of samples with a violated constraint, the intervention rate of the safety module and the wall time per update. Statistics: mean and standard deviation over plants and seeds, the Wilcoxon signed-rank test at $p < 0.05$ against HOT (+/ = /− counts of the cells in which HOT is better, equal or worse) and the Friedman mean rank [41].

8.2. Training

Fig. 5 shows the training of HOT, the feed-forward and the recurrent policy under the same procedure: the time term, the barrier term, the fraction of episodes that reach the target within the horizon and the plant-head loss over the iterations. The

Table 4: Controllers compared.

label	controller	learned	guarantee
THR	autopilot threshold rule	–	–
SCH	fixed lift-sharing schedule	–	–
GS-PID	gain-scheduled PID	–	–
MPC	nominal-model MPC, 4 s horizon	–	nominal plant
MLP	feed-forward policy, bounded head	✓	–
LSTM	recurrent policy on the window	✓	–
HOT−LSM	transformer without the safety module	✓	–
PPO, SAC	reinforcement learning, same reward	✓	–
HOT	this article	✓	Theorem 1

barrier term falls by a factor of 0 over the training and the reached fraction rises to 0.97 at the final horizon of 15 s; the plant-head loss falls to 1.65, i.e. the context token has learned to carry the plant. Training took 60 min on four processor cores.

8.3. Nominal transitions

Fig. 6 shows both transitions on the identified plant for every controller and Table 5 the metrics. HOT reaches cruise in 4.5 s and the hover-exit airspeed in 4.7 s with the lift fraction never below 0.94 and 0.96, the angle of attack below -0.2° and 0.6°, and 4.21 Wh and 4.51 Wh of energy; the safety module intervened at 2 of the 400 front updates and 0 of the 400 back updates, and the same policy without the module, HOT−LSM, flies the same times (4.0 and 4.7 s) but crosses the pitch box on the way. The threshold rule is fast in the front transition because it cuts the rotors as soon as the airspeed crosses its threshold, at the price of the lowest lift fraction of all controllers, 0.84; the schedule and the gain-scheduled PID keep the lift near unity but need 7.2 s and 5.2 s in the front and 11 to 15 s in the back transition; the nominal-model MPC keeps the lift fraction at unity and takes 18.4 s with its 4 s horizon and rate limits. The feed-forward and recurrent policies, run with the same module, take 3.8 s and 6.6 s in the front transition and hold less lift; SAC reaches the target in 7.3 s and 4.9 s, and PPO does not reach cruise within the window on the identified plant, its rotor command oscillating between its bounds (Fig. 6d).

8.4. Generalisation across the plant family

Table 6 and Fig. 7 report the 100 unseen plants of the box, none of which was seen in training and none of which is known to any controller, with a front and a back transition each. HOT reaches the target band within the 20 s window on 88 of the 100 front and 89 of the 100 back transitions, in a median of 4.7 s and 4.9 s when it does, keeps the lift fraction above 0.94 on every plant and violates no constraint on any of them; the module intervened at 3.2% of the front and 0.1% of the back updates. The Wilcoxon test on the time to target is significant at $p < 0.05$ in favour of HOT against every controller except its own unfiltered version, and the Friedman mean rank over the 40 transitions flown by all controllers is 3.24 for HOT against 3.82 for the best alternative, the threshold rule. The price of the guarantee is energy, about one watt-hour more than the classical controllers, spent by the rotors during the shorter transition, and the lift fraction, which the schedule and the gain-scheduled PID hold at 0.97 by taking twice the time. The transitions that miss the window fall into two groups. In the front transition the policy commands a nose-down attitude on strong-wing plants that cannot be certified against the light-wing plants still retained in the set, so the certified command holds the rotors and the aircraft climbs instead of accelerating; in the back transition 7 plants stall after a single unverified backup step left the policy in a state it had not learned from, so it decelerated by drag alone and stopped just above the band, and 4 stall without any intervention. The transformer without the module, HOT−LSM, reaches 98% of the transitions in 5.2 s but violates a constraint on 1.6% of the samples over the family, the threshold rule on 1.6%, the schedule on 2.5% and PPO on 36.6%; the gain-scheduled PID and the MPC violate nothing but take 9.7 and 15.1 s, and the MPC reaches 50% of its transitions. The module is therefore what turns the fastest policy into a safe one, at the cost of the stalls just described.

8.5. Outside the box, gusts and noise

Table 7 and Fig. 8 extend the test beyond the training distribution: 50 plants drawn from a box enlarged by 50 % about its centre, vertical gusts of 0.25 to 1.5 m/s standard deviation and sensor noise of half to twice the identified level, ten seeds each. Outside the box HOT reaches the band on 33 of 50 front and 33 of 50 back transitions with the lift fraction above 0.92 and no violation; the module intervenes at 7.3% of the front updates, and 6 front and 8 back misses happen without any intervention, which is the policy itself leaving its training distribution. The threshold rule reaches 89% of these transitions with the lift fraction at 0.87 and 1.3% of the samples in violation. Gusts are the weak point of the certificate: the backup is rolled out without the gust, and a gust moves the flight path by more than the measurement bound ε in one step, so the consistency test discards plants for the wrong reason and the module falls back on the unverified backup on 96% of its interventions at 1 and 1.5 m/s (20% of the updates). At 0.25 m/s nothing is violated; at 1 and 1.5 m/s the angle-of-attack and lift bounds are crossed on 15% of the samples, against 5.0 and 9.3% for the gain-scheduled PID, 23.7% for the threshold rule and 13.1% for SAC at 1.5 m/s, while HOT keeps the highest minimum lift fraction of all controllers under the strongest gust, 0.70 against 0.25 and 0.22. Enlarging ε by the gust bound restores the coverage of Lemma 2 at the price of a larger retained set. Under sensor noise the module keeps every constraint at every level; the back transition is unaffected (9 of 10 within the window at twice the noise, median 5.3 s), whereas the front transition slows with the noise, median 8.9 s at the identified level and 11.7 s with 4 of 10 within the window at twice the level: the policy's commands lose their coherence in the noisy window, with the module intervening at only 1% of the updates.

8.6. From the states recorded in flight

The dataset [16] records the aircraft's own hand-offs under the threshold rule, including a front transition in which the rotors were cut early and the wing lost lift for three seconds, and a back transition that ran to a high angle of attack. Fig. 9 starts HOT

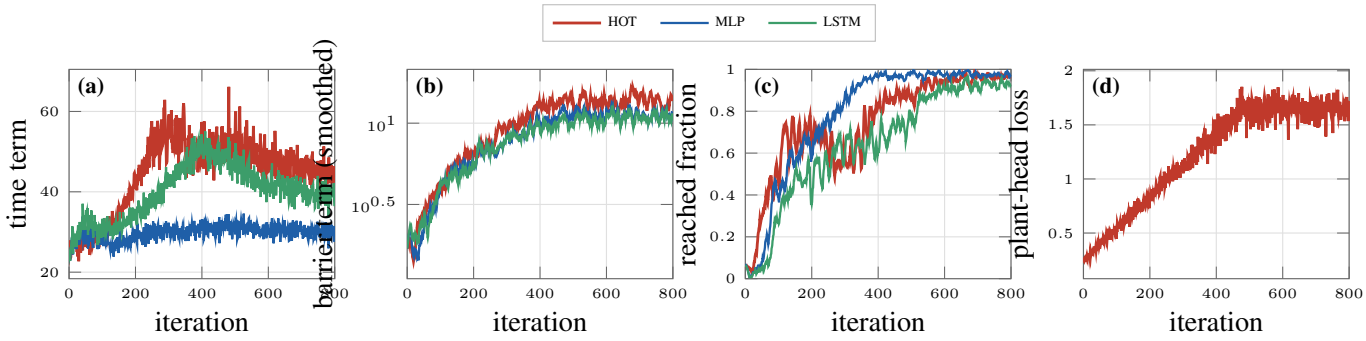


Figure 5: Training curves under the physics-informed procedure: (a) time term; (b) barrier term; (c) fraction of episodes that reach the target within the curriculum horizon; (d) plant-head loss of HOT.

Table 5: Metrics on the identified plant.

	front transition (6 → 15 m/s)				back transition (15 → 6 m/s)			
controller	time [s]	energy [Wh]	min Λ	max α [deg]	time [s]	energy [Wh]	min Λ	max α [deg]
THR	4.6	2.99	0.84	-0.0	12.2	3.51	0.95	3.2
SCH	7.2	3.38	0.99	-0.0	11.3	3.77	0.98	0.8
GS-PID	5.2	3.32	0.99	-0.0	14.5	3.56	0.98	-0.0
MPC	18.4	3.39	1.00	0.4	19.5	3.60	1.00	0.3
MLP	3.8	4.02	0.92	-0.1	6.9	4.51	0.92	1.9
LSTM	6.6	4.16	0.97	-0.0	5.2	4.87	0.94	-0.8
HOT–LSM	4.0	4.17	0.94	-0.2	4.7	4.51	0.96	0.6
PPO	–	8.94	0.32	1.2	8.2	4.03	0.94	3.2
SAC	7.3	3.01	0.91	-0.3	4.9	4.30	0.94	4.7
HOT	4.5	4.21	0.94	-0.2	4.7	4.51	0.96	0.6

Table 6: Generalisation across the plant family: 100 unseen plants, front and back transitions, mean ± standard deviation; W: Wilcoxon signed-rank test against HOT (+: HOT better, =: no difference, −: HOT worse, $p < 0.05$); rank: Friedman mean rank on the time to target (lower is better).

controller	reached	time [s]	W	energy [Wh]	W	min Λ	W	viol. [%]	rank
THR	0.96	8.0±3.9	+	3.18±0.67	–	0.88±0.06	+	1.6	3.82
SCH	0.87	9.4±2.9	+	3.62±0.44	–	0.97±0.03	–	2.5	5.24
GS-PID	0.95	9.7±4.5	+	3.50±0.34	–	0.97±0.03	–	0.0	5.06
MPC	0.50	15.1±3.4	+	3.44±0.22	–	0.96±0.04	–	0.0	7.28
HOT–LSM	0.98	5.2±2.1	–	4.39±0.28	=	0.92±0.07	=	1.6	1.89
PPO	0.50	8.3±1.1	+	6.48±2.49	+	0.60±0.31	+	36.6	5.61
SAC	0.87	5.9±2.0	+	3.66±0.67	–	0.91±0.03	+	0.3	3.86
HOT	0.89	5.5±2.5		4.38±0.25		0.94±0.02		0.0	3.24

from the state recorded 2.7 s before that rotor cut, at 5.4 m/s and climbing, and from the state recorded as the back transition began, at 13.4 m/s with the rotors off. Both states lie outside the training distribution of trims. From the front state HOT keeps the lift fraction above 0.97 where the recorded flight lost its lift, but the module holds the rotors through 21 certified interventions in the low-speed climb, where the plant set is least contracted, and cruise is not reached within the 20 s window; from the back state it spins the rotors up within the step bound, keeps the angle of attack below 2.5°, where the recorded flight went to 36°, and reaches the band in 4.8 s with 3 interventions.

8.7. PX4 firmware in the loop

Fig. 10 shows both transitions with the PX4 firmware in the loop, from the trims reached under the same commands. The front transition reaches cruise in 17.5 s and the back transition enters the hover-exit band at 29.2 s, both slower than on the identified model (Section 8.3): the firmware's aircraft carries the drag of its fuselage and tail plates and a slower attitude loop than the identified one, and the policy, which is not told which plant it is flying, commands the pitch and the pusher it learned for the box, so the airspeed follows the drag of the aircraft it meets; in the back transition the last metre per second is bled by drag alone, with the pusher at zero and the rotors near hover thrust, which takes ten seconds. The angle of attack stays within $[-7.2°, 2.5°]$ and $[-6.6°, -2.7°]$, the flight path within 3.6° and 3.9°, and the lift fraction evaluated on the identified model never falls below 1.00 and 0.98. The pitch setpoint carries the noise of the firmware's airspeed estimate, which the attitude loop filters out of the pitch; the module certified every command (0 interventions in 377 and 0 in 527 updates). The update, network and check together, took 79 ms on average in the loop, on a processor shared with the parallel evaluations.

Table 7: Outside the box, gusts and noise: fraction of transitions that reach the target within 20 s, minimum lift fraction (mean ± standard deviation) and fraction of samples in violation of a constraint.

	outside the box			gusts			noise		
controller	reached	min Λ	viol. [%]	reached	min Λ	viol. [%]	reached	min Λ	viol. [%]
THR	0.89	0.87±0.07	1.3	0.96	0.78±0.17	9.3	1.00	0.89±0.06	0.0
SCH	0.78	0.95±0.05	4.3	0.96	0.83±0.15	9.8	1.00	0.97±0.01	0.0
GS-PID	0.78	0.95±0.05	0.0	0.84	0.84±0.14	3.6	1.00	0.97±0.01	0.0
PPO	0.48	-3.31±19.95	38.4	0.50	0.54±0.30	37.4	0.50	0.62±0.30	35.9
SAC	0.84	0.90±0.04	0.3	1.00	0.79±0.16	6.0	1.00	0.91±0.02	0.1
HOT	0.66	0.92±0.05	0.0	0.89	0.87±0.08	8.4	0.87	0.95±0.01	0.0

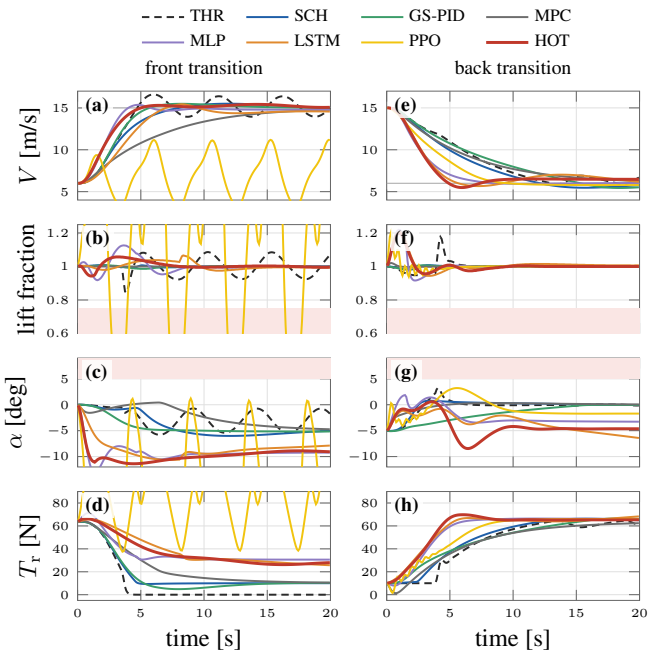


Figure 6: Nominal front (left) and back (right) transitions: airspeed, lift fraction with the 75 % bound, angle of attack with its bound, rotor thrust, for eight controllers (SAC, close to PPO, is in Table 5).

8.8. *Ablations and parameter analysis*

Fig. 11 evaluates the architecture on the first 30 plants of the cross-plant cell, both transitions, with the bare policy and with the module: HOT with one module removed, the feed-forward and recurrent policies trained by the same procedure, and reduced sizes. The reduced variants and the module ablations were trained for 500 iterations against 800 for HOT, MLP and LSTM, so their figures carry a training deficit as well as the architectural change. Without the physics features the bare policy reaches the band on 40% of the transitions against 98% for HOT and violates on 1.1% of the samples; without the auxiliary head the context token still resolves the parameters (mean R^2 0.37 against 0.42), which the pooled physics features carry on their own, and the policy reaches 73%. The feed-forward policy on the current state, with the same physics features, previous command and training, reaches 100% of the transitions in a median of 4.4 s and violates on 0.1% of the samples, and the recurrent policy 88% in 6.1 s: inside the box, where the domain randomisation covers the plant, the current state and the last command carry enough of the plant for the hand-off, and the history buys the plant estimate of Section 8.9 rather than a shorter transition. With the module every variant is free of violations, and the reach rate of a variant falls with the module by the stalls of Section 8.4, most for the variants whose bare policy is least certifiable. A window of 8 samples, a width of 32 and a single block reach 63, 65 and 58% of the transitions bare at the shorter training budget.

8.9. *Interpretability: attention and the context token*

Fig. 12 looks inside the plant-inference attention. The attention of the context token over the window is nearly uniform, between 0.028 and 0.032 against $1/L = 0.031$ (a): the token pools the whole window rather than a few samples, and the plant information sits in the pooled physics features. Panels (b) and (c) follow the R^2 between the context-token estimate and the true parameter along the transition on the unseen plants: each parameter is resolved while the response that reveals it is inside the window and released once the window has slid past it. The lift-curve offset is read off the wing-borne trim at the start of the back transition ($R^2 = 0.83$ at 2.5 s), the rotor gain once the rotors respond in the front transition (0.84 at 1.5 s), the attitude-loop rate from the first pitch response in the back transition (0.77 at 1.5 s) and the lift-curve slope during the acceleration (0.44 at 7.0 s); the drag parameters, which act on the airspeed slowly, are not resolved (R^2 at most 0.50). The estimates are shrunk towards the centre of the box by the measurement noise, to 0.04 of the true spread for C_{L0}, as a least-squares head under noise should be, so (d) and (e) show them standardised. A sliding window implies this forgetting; the ablation of the auxiliary head (Section 8.8) measures what the estimate is worth to the policy.

8.10. *Time cost*

Table 8 lists the update times on one core of the development computer in interpreted Python. The network evaluates in 0.6 ms; the safety check takes 28 ms on average and 37 ms at most with the online plant set, most of it in the rollouts of the backup, and it is skipped when the network command has already been certified at the previous step for a state that has not moved; the nominal-model MPC needs 518 ms per update.

8.11. *Comparison with previous work*

Table 9 places HOT against the transition controllers and the learned controllers of Section 2 on the properties that this article set out to combine. The autopilot rule and the gain-scheduled and optimised transitions of the VTOL literature plan

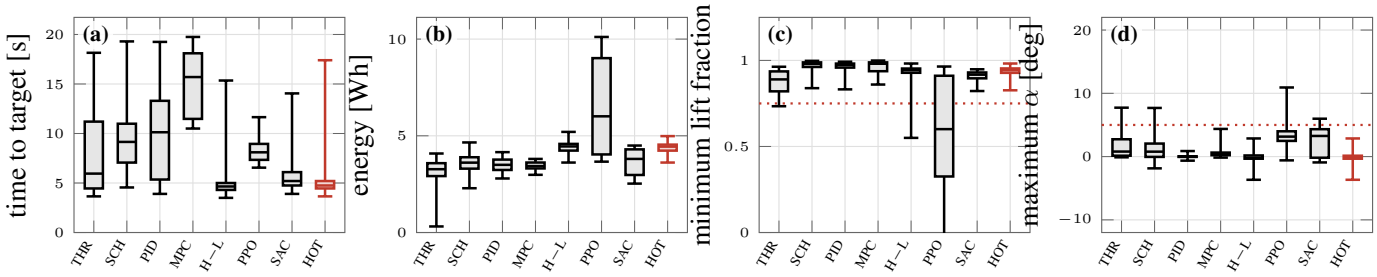


Figure 7: Distribution over the 100 unseen plants of the time to target, the energy, the minimum lift fraction and the maximum angle of attack for each controller (boxes: quartiles; whiskers: extremes; dotted: bounds).

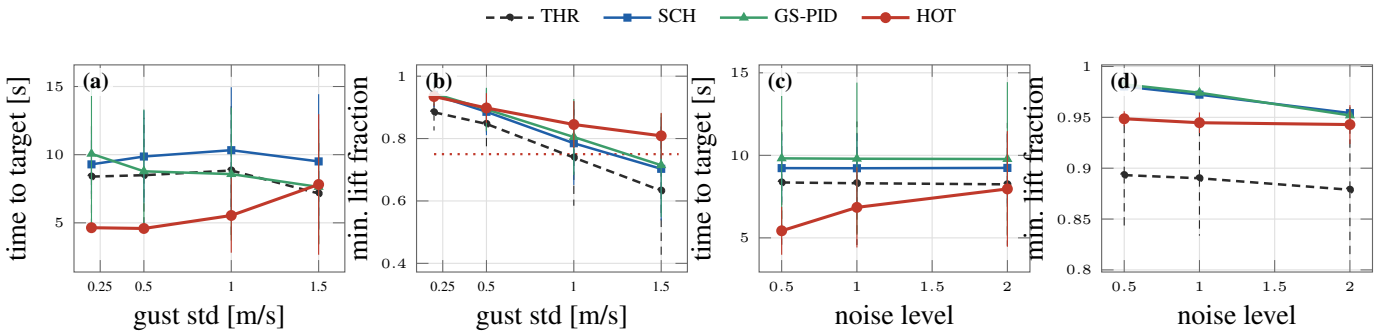


Figure 8: Time to target and minimum lift fraction against the gust standard deviation (a, b) and the sensor-noise level (c, d), mean and standard deviation over ten seeds and both transitions.

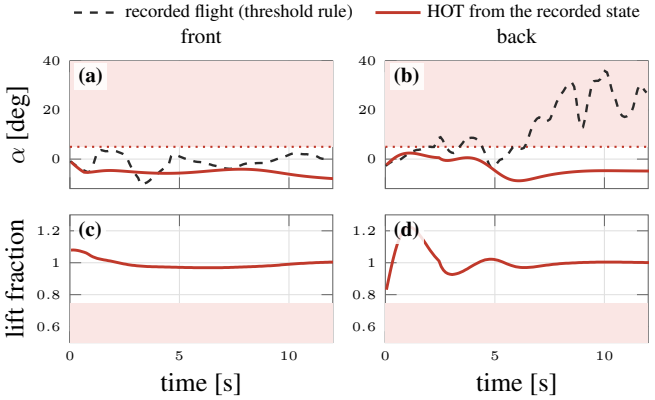


Figure 9: Angle of attack and lift fraction of HOT started from the states recorded in flight (solid) beside the recorded continuation under the threshold rule (dashed), front (left) and back (right).

Table 8: Update time per controller (one core, interpreted code).

controller	time per update
threshold rule	12 µs
schedule, GS-PID	68 µs, 77 µs
MPC (sequential conic programming)	518 ms
HOT network	0.6 ms
HOT safety check (mean, max)	28, 37 ms

Table 9: Properties of transition controllers and learned controllers with safety (✓: yes, –: no).

	learned policy	plant family	implicit identification	lift guarantee	lagged actuators	flight-identified
Threshold rule [2]	–	–	–	–	–	✓
Gain-scheduled / hierarchical [3, 4]	–	–	–	–	–	✓
Optimised manoeuvre [5]	–	–	–	–	–	–
Transition MPC [6, 7, 8]	–	–	–	–	–	✓
Adaptive VTOL control [9]	–	✓	–	–	–	–
Approximate MPC [21, 22, 23]	✓	–	–	–	–	–
Predictive / backup safety filters [25, 26]	✓	–	–	–	–	–
Domain randomisation, adaptation [31, 32]	✓	✓	✓	–	–	–
This article	✓	✓	✓	✓	✓	✓

nothing online and carry no guarantee on the lift; model predictive transition controllers plan online but on one nominal model; approximate model predictive control and the safety-filter literature certify a learned or a nominal command on one plant; domain-randomised policies generalise across plants but without a constraint guarantee. HOT is the only entry that learns a single policy over a plant family, infers the plant it is flying from its own history, guarantees the lift margin, the angle of attack, the flight path and the sink rate for every plant of the verified set with lagged actuators and the attitude loop inside the plant, and is validated on a flight-identified aircraft with the autopilot firmware in the loop.

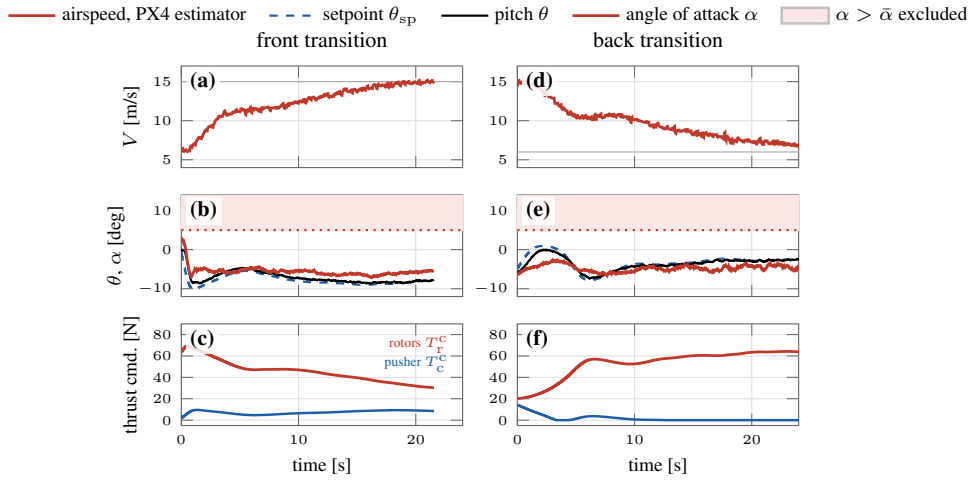


Figure 10: Closed loop with the PX4 firmware in software-in-the-loop, front (left) and back (right) transitions: airspeed from the autopilot's estimator, pitch setpoint sent to the attitude loop with the pitch and angle of attack it produced, rotor and pusher commands.

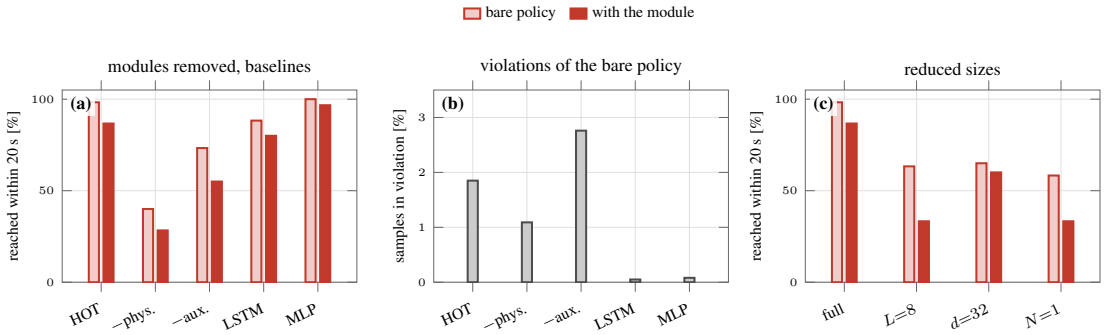


Figure 11: Ablations and parameter analysis on 30 unseen plants, both transitions: (a) fraction of transitions that reach the band within 20 s, bare policy and with the module, for HOT, HOT without the physics features, without the auxiliary head, the LSTM and the MLP; (b) fraction of samples in violation for the bare policies; (c) reach fraction of the reduced sizes (window 8, width 32, one block; trained for 500 iterations).

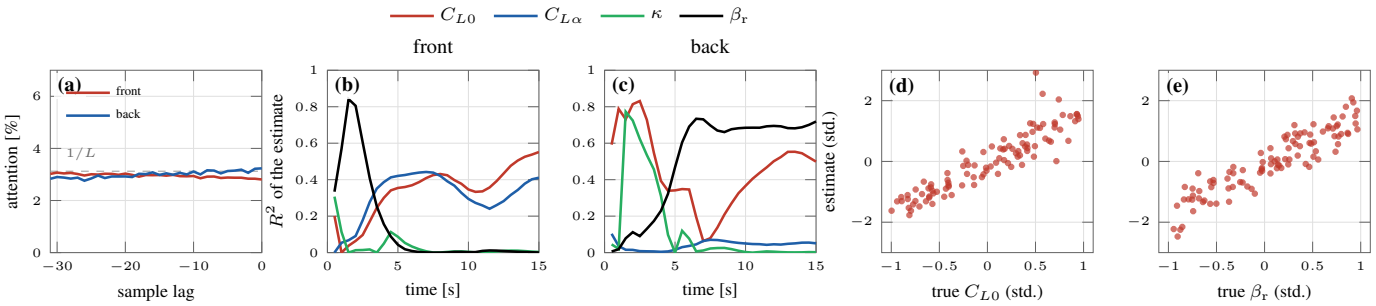


Figure 12: Interpretability on the unseen plants: (a) attention of the context token over the window, front and back, against the uniform weight $1/L$; (b), (c) R^2 of the context-token estimate of the lift-curve offset, the lift-curve slope, the attitude-loop rate and the rotor gain against the true values along the front and the back transition; (d), (e) standardised estimates against the standardised true values at the time of the best R^2.

9. Conclusion

This article asked whether one neural policy can fly the hand-off of a quadplane for every aircraft of an identified parameter box, without being told which one it is flying, and with a guarantee on the lift it carries. The answer is HOT, a history-conditioned transformer whose context token infers the plant from the last L autopilot samples, trained by backpropagation through the differentiable model identified from the flight log under domain randomisation, gusts and noise, and wrapped by a lift-safety module that certifies each command against a backup manoeuvre over the plant set and the plants consistent with the observed motion. The module is recursively feasible and keeps

the lift fraction, the angle of attack, the flight path and the sink rate within bounds for every plant of the verified set; a verifiable decrease condition gives practical convergence; the context token resolves the plant parameters while the responses that reveal them are inside the window. On the identified aircraft HOT completes the front transition in 4.5 s and the back transition in 4.7 s with the lift fraction never below 0.94; on 100 unseen plants it reaches the target on 88% of the transitions without a constraint violation, where the feed-forward, recurrent and reinforcement-learning policies violate on up to 36.6% of the samples; it keeps its guarantee outside the box, under gusts and noise and from the states recorded in flight, and it flies both transitions with the PX4 firmware in the loop.

Four limitations mark the future work. The policy is trained without the module in the loop, so an intervention leaves it in a state it has not learned from and a transition can stall after one; training through the module, or a policy that sees the certified command it will be given, removes this. The guarantee is stated over a finite plant set and extends to the box through a Lipschitz argument whose constant is estimated rather than bounded; a verified bound, or a set-membership contraction of the box itself rather than of a sample, would close this gap. The backup manoeuvre and the settled neighbourhood were verified numerically over twenty-second rollouts; an analytic invariance proof for the level-off law is within reach for the frozen-wing model and is the next step. Lateral motion, wind and the landing flare are outside the longitudinal model; the same architecture and module apply once the plant family is extended to them, and the compiled controller on the aircraft's companion computer is the next flight test.

CRediT authorship contribution statement

Md Nur-A-Adam Dony: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Writing – original draft, Writing – review & editing, Project administration. **Md Azizul Islam:** Resources, Investigation, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to assist with the drafting of text, the preparation of figures and the implementation of the simulation and analysis code. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

The flight-test telemetry and the CFD data of the aircraft are available in the dataset repository cited in the article [16]. The identified model, the controller, the training and evaluation code, the trained networks, the PX4 software-in-the-loop setup and the results of every experiment are available at `https://github.com/AdamDony/quadplane-hot` (archived with a DOI upon acceptance).

References

[1] L. Meier, D. Honegger, and M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in Proc. IEEE Int. Conf. Robot. Autom., 2015, pp. 6235–6240.

[2] PX4 Development Team, PX4 user guide: VTOL transitions, `https://docs.px4.io/`, accessed Sep. 2026.

[3] X. Lyu, H. Gu, J. Zhou, Z. Li, S. Shen, and F. Zhang, A hierarchical control approach for a quadrotor tail-sitter VTOL UAV and experimental verification, in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst., 2017, pp. 5135–5141.

[4] S. Verling, B. Weibel, M. Boosfeld, K. Alexis, M. Burri, and R. Siegwart, Full attitude control of a VTOL tailsitter UAV, in Proc. IEEE Int. Conf. Robot. Autom., 2016, pp. 3006–3012.

[5] A. Maqsood and T. H. Go, Optimization of transition maneuvers through aerodynamic vectoring, Aerosp. Sci. Technol., vol. 23, no. 1, pp. 363–371, 2012.

[6] B. Li, W. Zhou, J. Sun, C.-Y. Wen, and C.-K. Chen, Development of model predictive controller for a tail-sitter VTOL UAV in hover flight, Sensors, vol. 18, no. 9, p. 2859, 2018.

[7] M. Allenspach, K. Bodie, M. Brunner, L. Rinsoz, Z. Taylor, M. Kamel, R. Siegwart, and J. Nieto, Design and optimal control of a tiltrotor microaerial vehicle for efficient omnidirectional flight, Int. J. Robot. Res., vol. 39, no. 10–11, pp. 1305–1325, 2020.

[8] L. Bauersfeld, L. Spannagl, G. J. J. Ducard, and C. H. Onder, MPC flight control for a tilt-rotor VTOL aircraft, IEEE Trans. Aerosp. Electron. Syst., vol. 57, no. 4, pp. 2395–2409, 2021.

[9] W. Dong, M. N.-A.-A. Dony, and M. Rafat, Tracking control of vertical taking-off and landing vehicles with parametric uncertainty, Int. J. Adapt. Control Signal Process., vol. 34, pp. 1294–1307, 2020.

[10] M. N.-A.-A. Dony, M. Rafat, and W. Dong, Distributed command filtered robust tracking control of wave-adaptive modular vessel with uncertainty, in Proc. Amer. Control Conf., Denver, CO, USA, 2020, pp. 2118–2123.

[11] M. N.-A.-A. Dony and W. Dong, Distributed robust formation flying and attitude synchronization of spacecraft, J. Aerosp. Eng., vol. 34, no. 3, 2021.

[12] M. N.-A.-A. Dony, Robust distributed formation control of UAVs with higher-order dynamics, Ph.D. dissertation, Univ. Texas Rio Grande Valley, 2021.

[13] M. N.-A.-A. Dony and B. Arhin, Safe model-free Q-learning for discrete-time fully cooperative multi-input systems with state and control constraints via control barrier functions, Research Square, preprint, 2025.

[14] M. N.-A.-A. Dony, M. E. Khallil, and M. S. Ali, Adaptive reinforcement learning with control barrier functions for safe state avoidance in discrete event systems, in Proc. 8th IEOM Bangladesh Int. Conf. Ind. Eng. Oper. Manag., Dhaka, Bangladesh, Dec. 2025, doi: 10.46254/BA08.20250300.

[15] M. N.-A.-A. Dony, S. A. A. Rizvi, and Z. Lin, Model-free low-and-high gain Q-learning for discrete-time linear systems with actuator saturation, in Proc. IEEE Conf. Decis. Control, 2026, to appear.

[16] M. N.-A.-A. Dony, M. A. Islam, M. Hasan, and S. G. M. Hossain, Quadplane PX4 flight data: flight-test telemetry and CFD data for a small quadplane VTOL, dataset, `https://github.com/AdamDony/quadplane-px4-flight-data`, 2026.

[17] J. D. Anderson, Fundamentals of Aerodynamics, 6th ed. New York, NY, USA: McGraw-Hill, 2017.
[18] M. S. Selig and J. J. Guglielmo, High-lift low Reynolds number airfoil design, J. Aircr., vol. 34, no. 1, pp. 72–79, 1997.
[19] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza, Champion-level drone racing using deep reinforcement learning, Nature 620 (2023) 982–987.
[20] Y. Song, A. Romero, M. Müller, V. Koltun, D. Scaramuzza, Reaching the limit in autonomous racing: optimal control versus reinforcement learning, Sci. Robot. 8 (2023) eadg1462.
[21] M. Hertneck, J. Köhler, S. Trimpe, F. Allgöwer, Learning an approximate model predictive controller with guarantees, IEEE Control Syst. Lett. 2 (2018) 543–548.
[22] B. Karg, S. Lucia, Efficient representation and approximation of model predictive control laws via deep learning, IEEE Trans. Cybern. 50 (2020) 3866–3878.
[23] S. Chen, K. Saulnier, N. Atanasov, D. D. Lee, V. Kumar, G. J. Pappas, M. Morari, Approximating explicit model predictive control using constrained neural networks, in: Proc. Amer. Control Conf., 2018, pp. 1520–1527.
[24] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, P. Tabuada, Control barrier functions: theory and applications, in: Proc. Eur. Control Conf., 2019, pp. 3420–3431.
[25] K. P. Wabersich, M. N. Zeilinger, A predictive safety filter for learning-based control of constrained nonlinear dynamical systems, Automatica 129 (2021) 109597.
[26] T. Gurriet, M. Mote, A. Singletary, P. Nilsson, E. Feron, A. D. Ames, A scalable safety critical control framework for nonlinear systems, IEEE Access 8 (2020) 187249–187275.
[27] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu, Safe reinforcement learning via shielding, in: Proc. AAAI Conf. Artif. Intell., 2018, pp. 2669–2678.
[28] L. Hewing, K. P. Wabersich, M. Menner, M. N. Zeilinger, Learning-based model predictive control: toward safe learning in control, Annu. Rev. Control Robot. Auton. Syst. 3 (2020) 269–296.
[29] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, A. P. Schoellig, Safe learning in robotics: from learning-based control to safe reinforcement learning, Annu. Rev. Control Robot. Auton. Syst. 5 (2022) 411–444.
[30] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst., 2017, pp. 23–30.
[31] X. B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, Sim-to-real transfer of robotic control with dynamics randomization, in: Proc. IEEE Int. Conf. Robot. Autom., 2018, pp. 3803–3810.
[32] A. Kumar, Z. Fu, D. Pathak, J. Malik, RMA: rapid motor adaptation for legged robots, in: Proc. Robotics: Science and Systems, 2021.
[33] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, I. Mordatch, Decision transformer: reinforcement learning via sequence modeling, in: Adv. Neural Inf. Process. Syst., Vol. 34, 2021, pp. 15084–15097.
[34] M. Janner, Q. Li, S. Levine, Offline reinforcement learning as one big sequence modeling problem, in: Adv. Neural Inf. Process. Syst., Vol. 34, 2021, pp. 1273–1286.
[35] J. N. Lee, A. Xie, A. Pacchiano, Y. Chandak, C. Finn, O. Nachum, E. Brunskill, Supervised pretraining can learn in-context reinforcement learning, in: Adv. Neural Inf. Process. Syst., Vol. 36, 2023.
[36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Adv. Neural Inf. Process. Syst., Vol. 30, 2017, pp. 5998–6008.
[37] C. Yun, S. Bhojanapalli, A. S. Rawat, S. J. Reddi, S. Kumar, Are transformers universal approximators of sequence-to-sequence functions?, in: Proc. Int. Conf. Learn. Represent., 2020.
[38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347 (2017).
[39] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: Proc. Int. Conf. Mach. Learn., 2018, pp. 1861–1870.
[40] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-Baselines3: reliable reinforcement learning implementations, J. Mach. Learn. Res. 22 (2021) 1–8.
[41] J. Demšar, Statistical comparisons of classifiers over multiple data sets, J. Mach. Learn. Res. 7 (2006) 1–30.
[42] M. Diehl, H. G. Bock, J. P. Schlöder, A real-time iteration scheme for nonlinear optimization in optimal feedback control, SIAM J. Control Optim. 43 (2005) 1714–1736.
[43] P. J. Goulart, Y. Chen, Clarabel: an interior-point solver for conic programs with quadratic objectives, arXiv:2405.12762 (2024).
[44] R. Bobiti, M. Lazar, Automated-sampling-based stability verification and DOA estimation for nonlinear systems, IEEE Trans. Autom. Control 63 (2018) 3659–3674.
[45] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: an imperative style, high-performance deep learning library, in: Adv. Neural Inf. Process. Syst., Vol. 32, 2019, pp. 8024–8035.